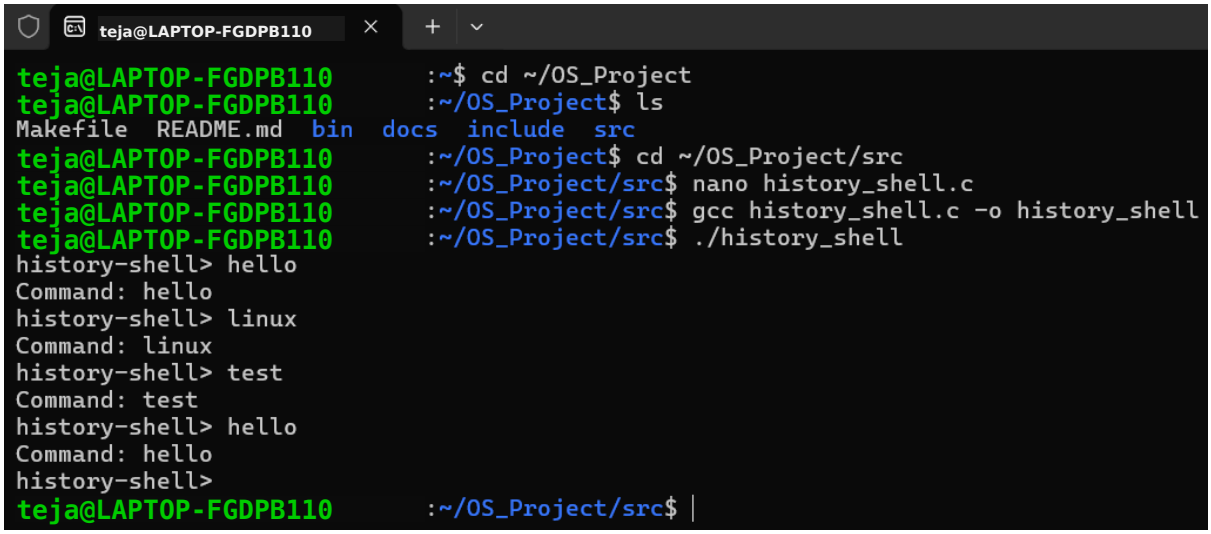
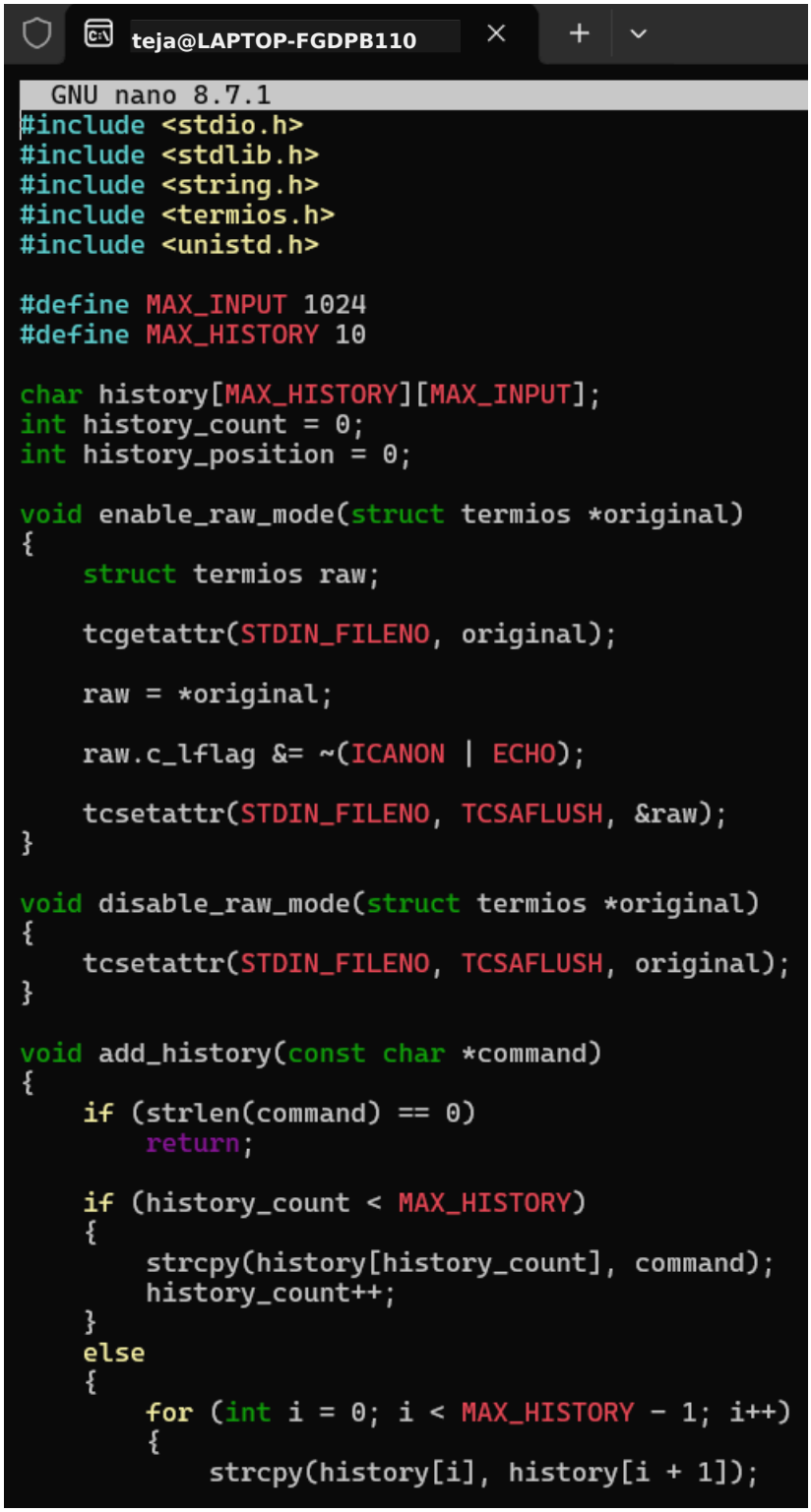
Skill Week-2

Task 1: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.




teja@LAPTOP-FGDPB110
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>
#define MAX_INPUT 1024
#define MAX_HISTORY 10
char history[MAX_HISTORY][MAX_INPUT];
int history_count = 0;
int history_position = 0;
void enable_raw_mode(struct termios *original)
{
struct termios raw;
tcgetattr(STDIN_FILENO, original);
raw = *original;
raw.c_lflag &= ~(ICANON | ECHO);
tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}
void disable_raw_mode(struct termios *original)
{
tcsetattr(STDIN_FILENO, TCSAFLUSH, original);
}
void add_history(const char *command)
{
if (strlen(command) == 0)
return;
if (history_count < MAX_HISTORY)
{
strcpy(history[history_count], command);
history_count++;
}
else
{
for (int i = 0; i < MAX_HISTORY - 1; i++)
{
strcpy(history[i], history[i + 1]);

```
        strcpy(history[MAX_HISTORY - 1], command);
    }

    history_position = history_count;
}

void clear_line(int length)
{
    for (int i = 0; i < length; i++)
    {
        printf("\b \b");
    }
}

int main()
{
    struct termios original;

    char buffer[MAX_INPUT];
    int position = 0;
    int displayed_length = 0;

    char ch;

    enable_raw_mode(&original);

    while (1)
    {
        position = 0;
        displayed_length = 0;
        buffer[0] = '\0';

        printf("history-shell> ");
        fflush(stdout);

        while (1)
        {
            if (read(STDIN_FILENO, &ch, 1) <= 0)
                break;

            /* Enter */
            if (ch == '\n' || ch == '\r')
            {
                buffer[position] = '\0';
```

```
            printf("\n");

            if (strcmp(buffer, "exit") == 0)
            {
                disable_raw_mode(&original);
                printf("Shell terminated.\n");
                return 0;
            }

            if (strlen(buffer) > 0)
            {
                printf("Command: %s\n", buffer);
                add_history(buffer);
            }

            break;
        }

        /* Backspace */
        else if (ch == 127 || ch == '\b')
        {
            if (position > 0)
            {
                position--;
                buffer[position] = '\0';

                printf("\b \b");
                displayed_length--;
                fflush(stdout);
            }
        }

        /* Escape sequence */
        else if (ch == 27)
        {
            char seq[2];

            if (read(STDIN_FILENO, &seq[0], 1) <= 0)
                continue;

            if (seq[0] != '[')
                continue;

            if (read(STDIN_FILENO, &seq[1], 1) <= 0)
                continue;
```

```
    if (seq[1] == 'A')
    {
        if (history_count > 0 && history_position > 0)
        {
            clear_line(displayed_length);

            history_position--;

            strcpy(buffer, history[history_position]);

            position = strlen(buffer);
            displayed_length = position;

            printf("%s", buffer);
            fflush(stdout);
        }
    }

    /* Down Arrow */
    else if (seq[1] == 'B')
    {
        if (history_position < history_count - 1)
        {
            clear_line(displayed_length);

            history_position++;

            strcpy(buffer, history[history_position]);

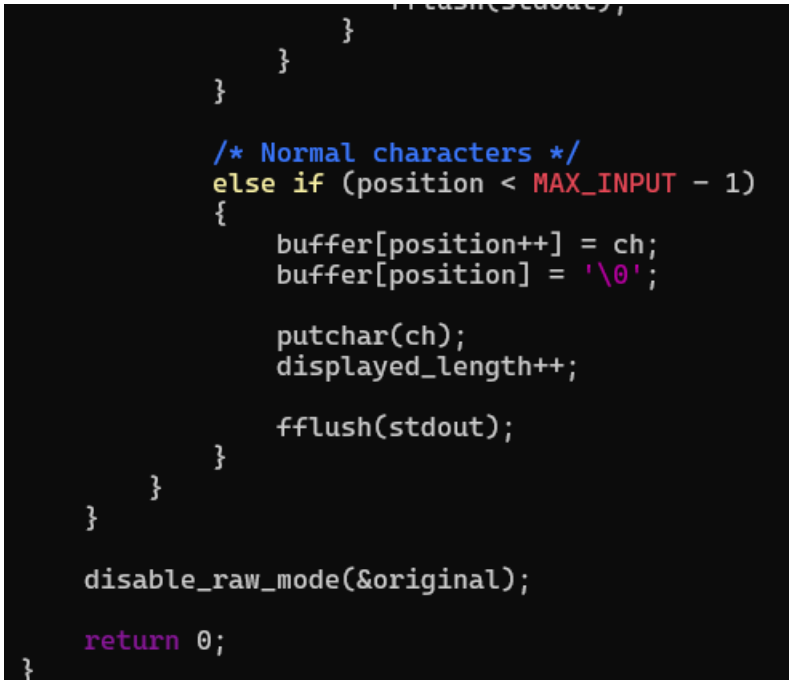
            position = strlen(buffer);
            displayed_length = position;

            printf("%s", buffer);
            fflush(stdout);
        }
        else if (history_position == history_count - 1)
        {
            clear_line(displayed_length);

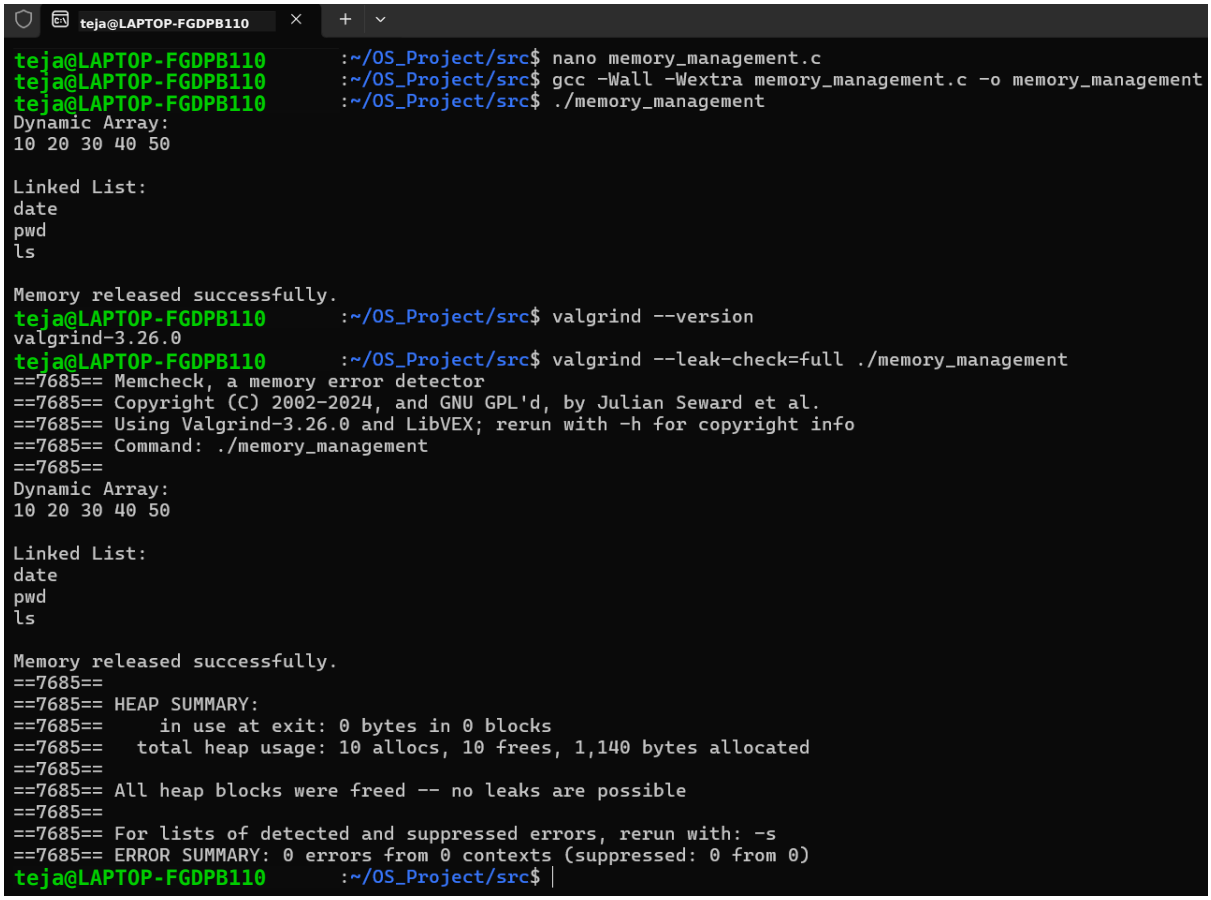
            history_position = history_count;

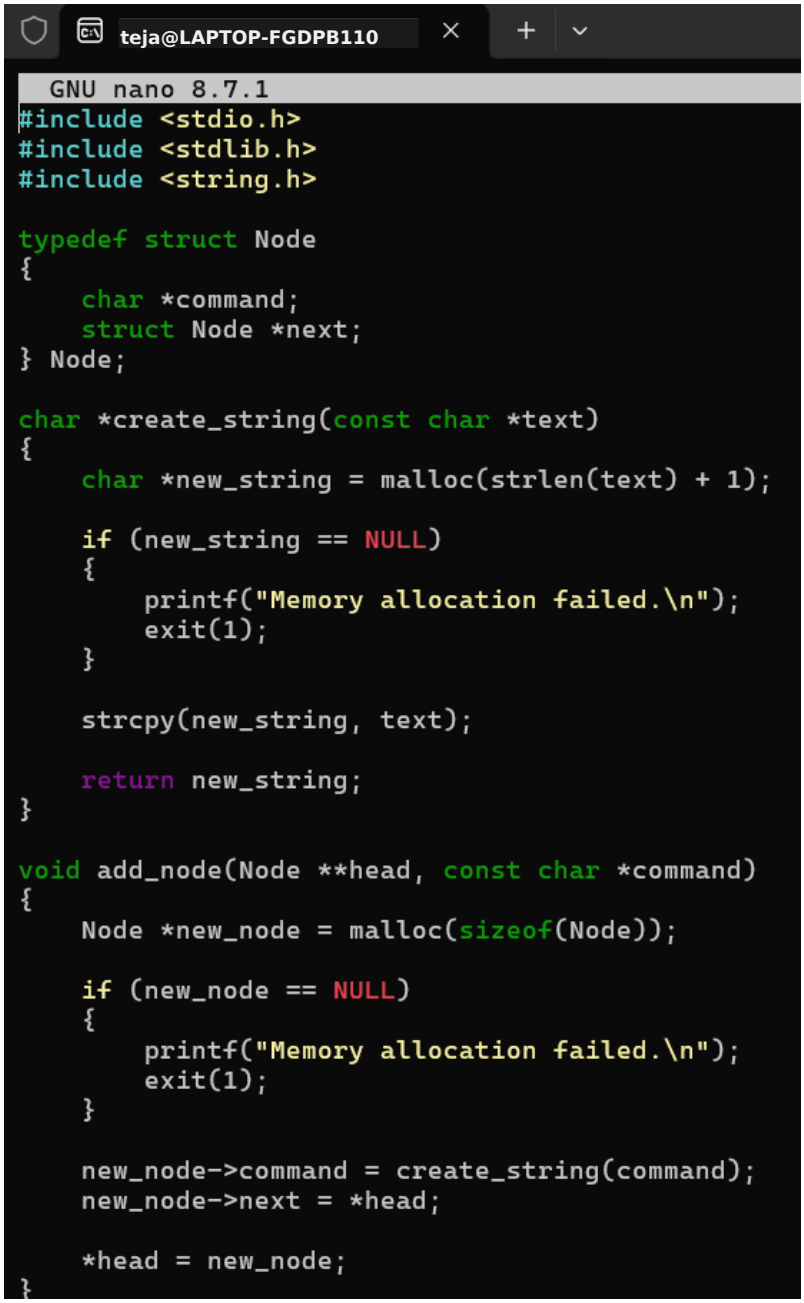
            position = 0;
            displayed_length = 0;
            buffer[0] = '\0';

           fflush(stdout);
```



Task 2: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.


teja@LAPTOP-FGDPB110
teja@LAPTOP-FGDPB110 :~/OS_Project/src$ nano memory_management.c
teja@LAPTOP-FGDPB110 :~/OS_Project/src$ gcc -Wall -Wextra memory_management.c -o memory_management
teja@LAPTOP-FGDPB110 :~/OS_Project/src$ ./memory_management
Dynamic Array:
10 20 30 40 50
Linked List:
date
pwd
ls
Memory released successfully.
teja@LAPTOP-FGDPB110 :~/OS_Project/src$ valgrind --version
valgrind-3.26.0
teja@LAPTOP-FGDPB110 :~/OS_Project/src$ valgrind --leak-check=full ./memory_management
==7685== Memcheck, a memory error detector
==7685== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==7685== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==7685== Command: ./memory_management
==7685==
Dynamic Array:
10 20 30 40 50
Linked List:
date
pwd
ls
Memory released successfully.
==7685==
==7685== HEAP SUMMARY:
==7685== in use at exit: 0 bytes in 0 blocks
==7685== total heap usage: 10 allocs, 10 frees, 1,140 bytes allocated
==7685==
==7685== All heap blocks were freed -- no leaks are possible
==7685==
==7685== For lists of detected and suppressed errors, rerun with: -s
==7685== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
teja@LAPTOP-FGDPB110 :~/OS_Project/src$


teja@LAPTOP-FGDPB110
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node
{
char *command;
struct Node *next;
} Node;
char *create_string(const char *text)
{
char *new_string = malloc(strlen(text) + 1);
if (new_string == NULL)
{
printf("Memory allocation failed.\n");
exit(1);
}
strcpy(new_string, text);
return new_string;
}
void add_node(Node **head, const char *command)
{
Node *new_node = malloc(sizeof(Node));
if (new_node == NULL)
{
printf("Memory allocation failed.\n");
exit(1);
}
new_node->command = create_string(command);
new_node->next = *head;
*head = new_node;
}

```c
void print_list(Node *head)
{
    Node *current = head;

    printf("\nLinked List:\n");

    while (current != NULL)
    {
        printf("%s\n", current->command);
        current = current->next;
    }
}

void free_list(Node *head)
{
    Node *current = head;

    while (current != NULL)
    {
        Node *temp = current;

        current = current->next;

        free(temp->command);
        free(temp);
    }
}

int main()
{
    int capacity = 2;
    int size = 0;

    int *numbers = malloc(capacity * sizeof(int));

    if (numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }
```

```c
    printf("Dynamic Array:\n");

    for (int i = 1; i <= 5; i++)
    {
        if (size == capacity)
        {
            capacity *= 2;

            int *temp = realloc(numbers, capacity * sizeof(int));

            if (temp == NULL)
            {
                free(numbers);
                printf("Reallocation failed.\n");
                return 1;
            }

            numbers = temp;
        }

        numbers[size++] = i * 10;
    }

    for (int i = 0; i < size; i++)
    {
        printf("%d ", numbers[i]);
    }

    printf("\n");

    Node *head = NULL;

    add_node(&head, "ls");
    add_node(&head, "pwd");
    add_node(&head, "date");

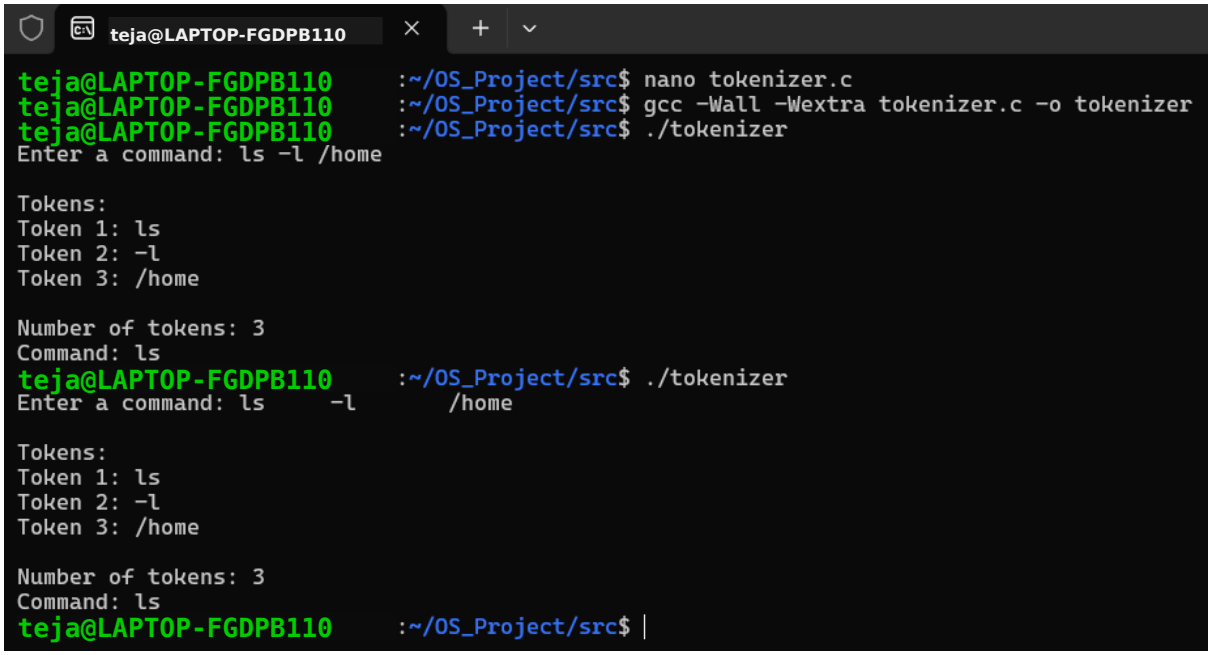
    print_list(head);

    free_list(head);
    free(numbers);

    printf("\nMemory released successfully.\n");

    return 0;
}
```

Task 3: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.


teja@LAPTOP-FGDPB110
teja@LAPTOP-FGDPB110 :~/OS_Project/src$ nano tokenizer.c
teja@LAPTOP-FGDPB110 :~/OS_Project/src$ gcc -Wall -Wextra tokenizer.c -o tokenizer
teja@LAPTOP-FGDPB110 :~/OS_Project/src$ ./tokenizer
Enter a command: ls -l /home
Tokens:
Token 1: ls
Token 2: -l
Token 3: /home
Number of tokens: 3
Command: ls
teja@LAPTOP-FGDPB110 :~/OS_Project/src$ ./tokenizer
Enter a command: ls -l /home
Tokens:
Token 1: ls
Token 2: -l
Token 3: /home
Number of tokens: 3
Command: ls
teja@LAPTOP-FGDPB110 :~/OS_Project/src$

teja@LAPTOP-FGDPB110

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

typedef struct
{
    char *value;
} Token;

int tokenize(char *input, Token tokens[])
{
    int count = 0;

    char *token = strtok(input, " \t\n");

    while (token != NULL && count < MAX_TOKENS)
    {
        tokens[count].value = token;
        count++;

        token = strtok(NULL, " \t\n");
    }

    return count;
}

int main()
{
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Enter a command: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    int count = tokenize(input, tokens);

    if (count == 0)
    {
        printf("No tokens found.\n");
```

```c
    if (count == 0)
    {
        printf("No tokens found.\n");
        return 0;
    }

    printf("\nTokens:\n");

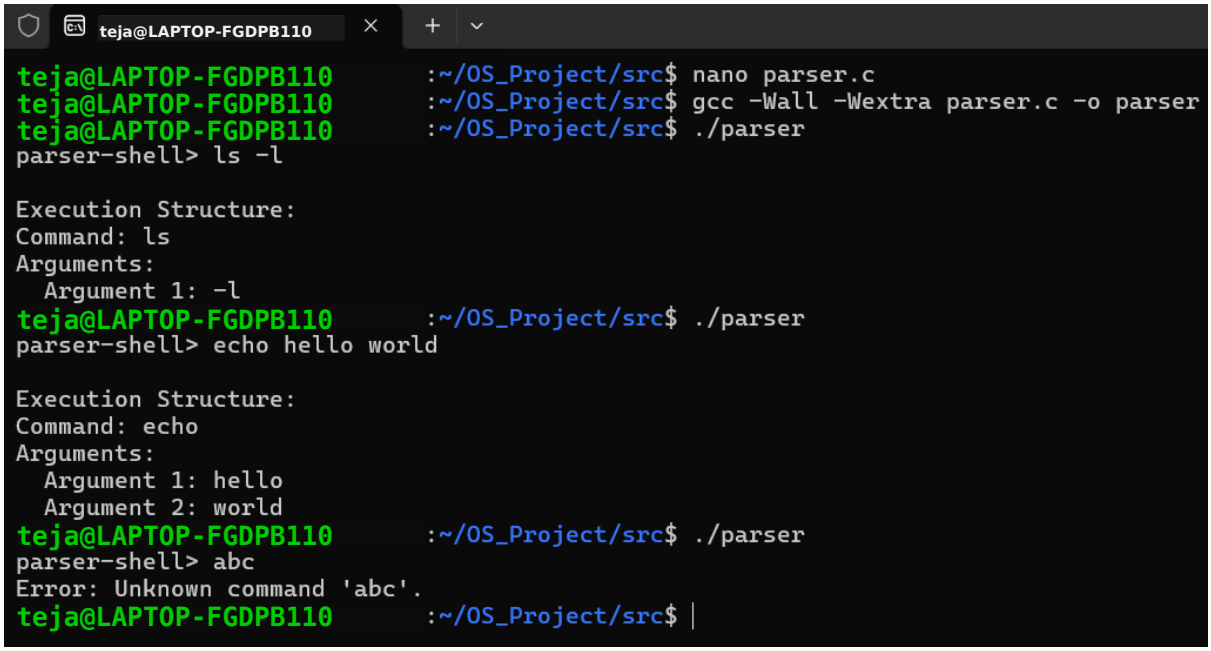
    for (int i = 0; i < count; i++)
    {
        printf("Token %d: %s\n", i + 1, tokens[i].value);
    }

    printf("\nNumber of tokens: %d\n", count);

    if (strcmp(tokens[0].value, "exit") == 0)
    {
        printf("Valid exit command.\n");
    }
    else
    {
        printf("Command: %s\n", tokens[0].value);
    }

    return 0;
}
```

Task 4: To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.



```
 GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50

typedef struct
{
    char *args[MAX_ARGS];
    int argc;
} Command;

int parse_command(char *input, Command *cmd)
{
    cmd->argc = 0;

    char *token = strtok(input, " \t\n");

    while (token != NULL)
    {
        if (cmd->argc >= MAX_ARGS - 1)
        {
            printf("Error: Too many arguments.\n");
            return 0;
        }

        cmd->args[cmd->argc] = token;
        cmd->argc++;

        token = strtok(NULL, " \t\n");
    }

    cmd->args[cmd->argc] = NULL;

    return 1;
}

int validate_command(Command *cmd)
{
    if (cmd->argc == 0)
    {
        printf("Error: Empty command.\n");
        return 0;
    }
```

```
    if (strcmp(cmd->args[0], "exit") == 0)
    {
        return 1;
    }

    if (strcmp(cmd->args[0], "ls") == 0 ||
        strcmp(cmd->args[0], "pwd") == 0 ||
        strcmp(cmd->args[0], "echo") == 0)
    {
        return 1;
    }

    printf("Error: Unknown command '%s'.\n", cmd->args[0]);

    return 0;
}

void print_execution_structure(Command *cmd)
{
    printf("\nExecution Structure:\n");

    printf("Command: %s\n", cmd->args[0]);

    printf("Arguments:\n");

    for (int i = 1; i < cmd->argc; i++)
    {
        printf("  Argument %d: %s\n", i, cmd->args[i]);
    }
}

int main()
{
    char input[MAX_INPUT];

    Command cmd;

    printf("parser-shell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }
```

```
    }

    if (!parse_command(input, &cmd))
    {
        return 1;
    }

    if (!validate_command(&cmd))
    {
        return 1;
    }

    if (strcmp(cmd.args[0], "exit") == 0)
    {
        printf("Exit command detected.\n");
        return 0;
    }

    print_execution_structure(&cmd);

    return 0;
}
```